MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Design Analysis and Algorithm
Exp. 7

Name : Shubham Vijaykumar Nikwade

Roll no : 17

Batch : A1

PRN : 202401120033

❖ Design and Implement application based on back-tracking approach -N Queens Problem

● Theory:

The N-Queens problem is a chessboard problem where we have to place N queens on an N × N chessboard such that no two queens can attack each other.

Two queens should not be in the same:

- Row
- Column
- Diagonal

It is solved using Backtracking, where we place queens one by one. If a position is not safe, we try another position. If no position is possible, we go back and change the previous queen's position. This continues until a valid solution is found.

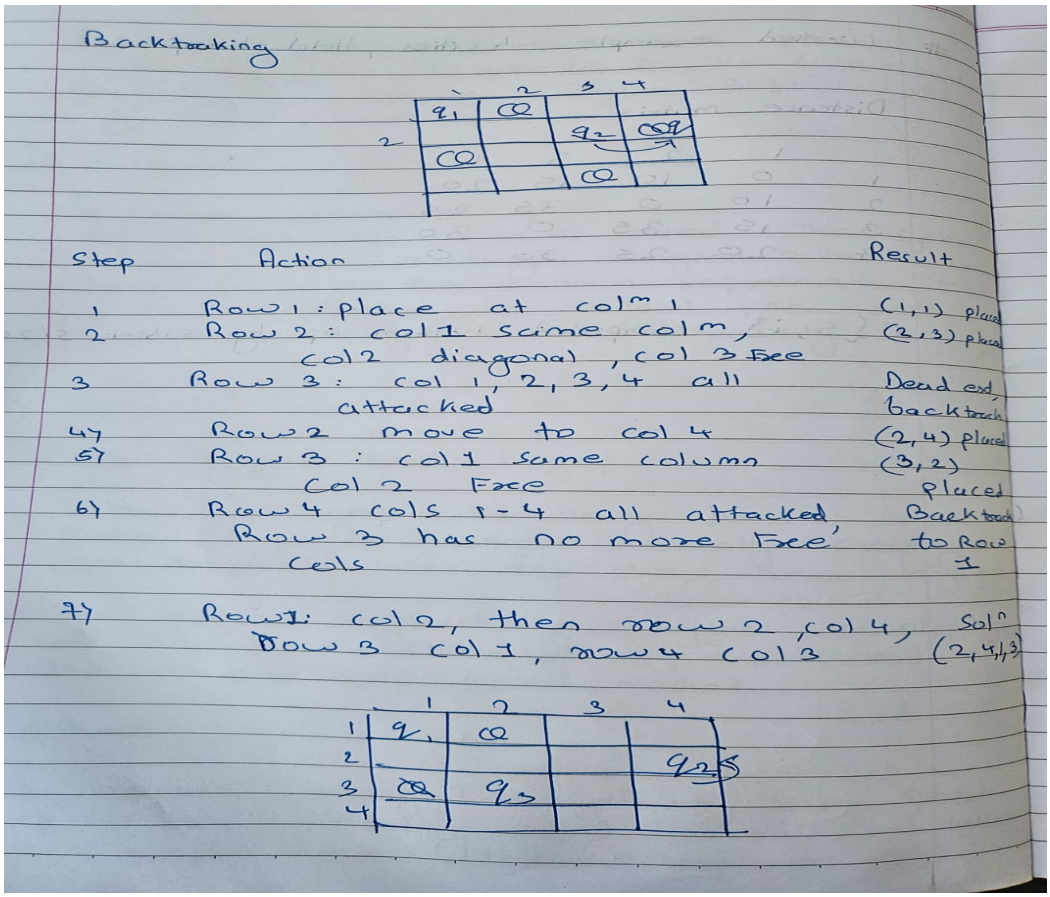
For N-Queens, we place one queen in each row and try different columns.

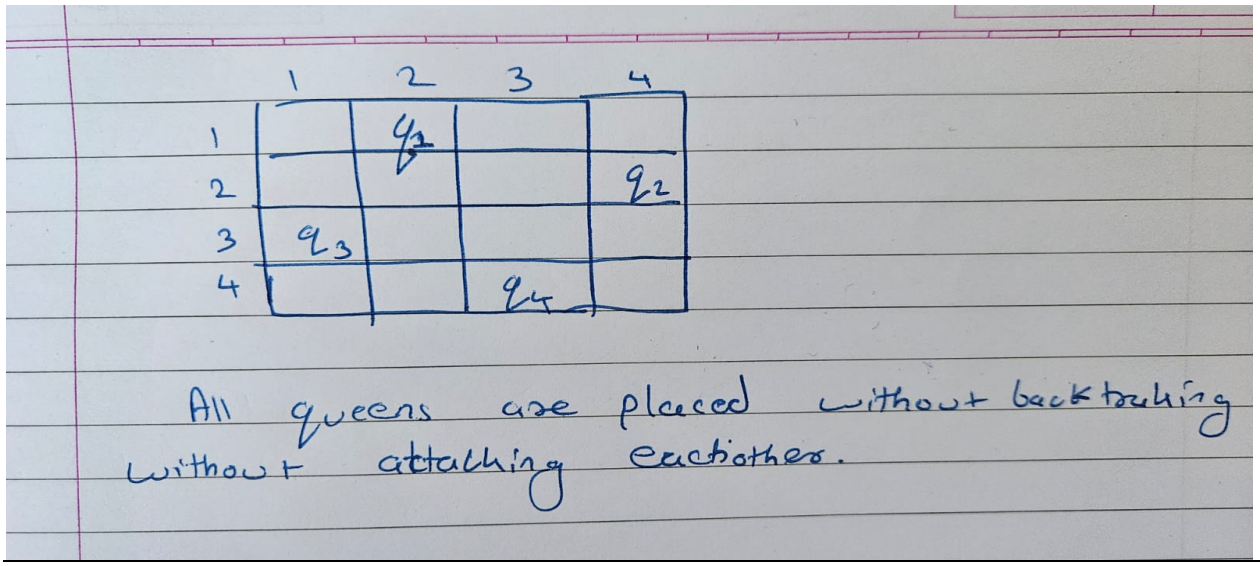
● Algorithm:

1. Start from the first row.
2. Try placing a queen in every column.
3. Check whether the position is safe.
4. If the position is safe:
    1. Place the queen.
    2. Move to the next row.
5. If all rows are completed, a solution is found.
6. If no safe position is available:

    1. Remove the previously placed queen.
    2. Try another position.
7. Repeat until a solution is found or all possibilities are checked.

- **Example:**


Backtracking
Step
Action
Result
1 Row 1 : place at colm 1 (1,1) placed
2 Row 2 : col 1 same colm, col 2 diagonal, col 3 Free (2,3) placed
3 Row 3 : col 1, 2, 3, 4 all attacked Dead end, backtrack
4) Row 2 move to col 4 (2,4) placed
5) Row 3 : col 1 same column col 2 Free (3,2) placed
6) Row 4 cols 1-4 all attacked, Row 3 has no more Free cols Backtrack to Row 1
7) Row 1: col 2, then row 2, col 4, Row 3 col 1, row 4 col 3 Sol^n (2,4,1,3)


All queens are placed without backtracking without attacking eachothers.

Code:

```
#include <iostream>
using namespace std;

int board[10][10];
int n;

bool isSafe(int row, int col)
{
   for (int i = 0; i < row; i++)
   {
      if (board[i][col] == 1)
         return false;
   }

   for (int i = row - 1, j = col - 1;
       i >= 0 && j >= 0; i--, j--)
   {
      if (board[i][j] == 1)
         return false;
   }

   for (int i = row - 1, j = col + 1;
       i >= 0 && j < n; i--, j++)
   {
      if (board[i][j] == 1)
         return false;
   }

   return true;
}

bool solveNQueens(int row)
{
   if (row == n)
      return true;

   for (int col = 0; col < n; col++)
```

```
    {
        if (isSafe(row, col))
        {
            board[row][col] = 1;

            if (solveNQueens(row + 1))
                return true;

            // Backtracking
            board[row][col] = 0;
        }
    }

    return false;
}

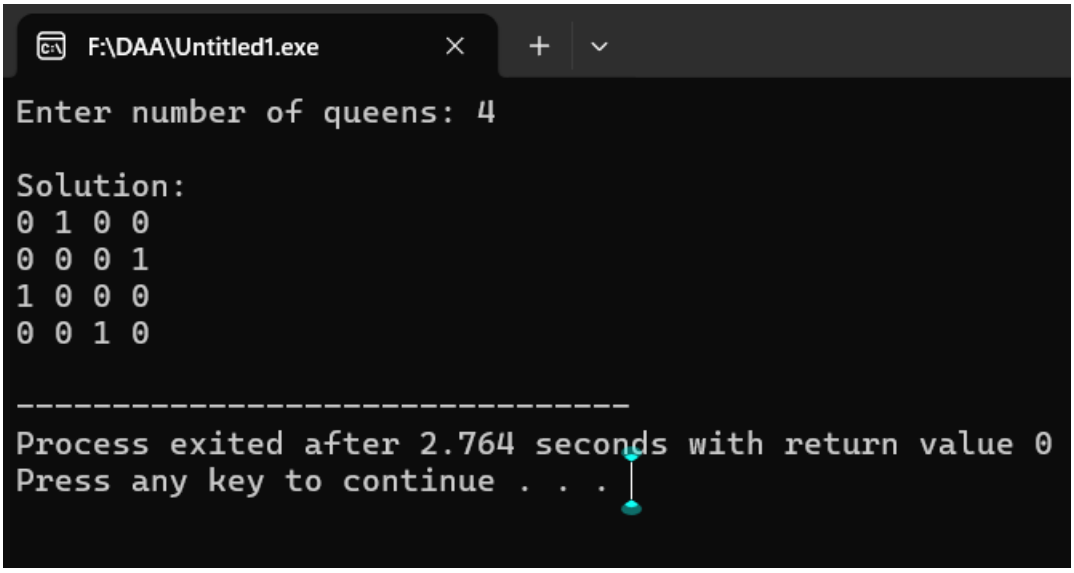
int main()
{
    cout << "Enter number of queens: ";
    cin >> n;

    if (solveNQueens(0))
    {
        cout << "\nSolution:\n";

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                cout << board[i][j] << " ";
            }
            cout << endl;
        }
    }
    else
    {
        cout << "No solution exists.";
    }
```

```
    return 0;
}
```

Output:



- **Time Complexity Derivation:**

In the worst case, we try many possible arrangements of queens.

For the first row, there can be N choices.

For the second row, approximately N-1 choices.

For the third row, approximately N-2 choices.

Therefore:

$N \times (N-1) \times (N-2) \times \ldots \times 1$

So the time complexity is approximately:

$O(N!)$

- **Space Complexity:**

We use an N × N chessboard.

Therefore, space required for the board is:

$N \times N = N^2$

The recursive function also uses a recursion stack of at most N.

So the overall space complexity is:

$O(N^2)$

Applications

1. Chessboard arrangement problems
2. Scheduling problems
3. Resource allocation
4. Constraint satisfaction problems
5. Combinatorial problem solving
